

MANUAL DE USUARIO

Implementación de PyC45 con Nuevos Datasets

Framework de Clasificación C4.5 en Python

Algoritmos de Construcción, Poda y Evaluación de Modelos

Autor: Luis Alberto Buelvas Cogollo

Framework: PyC45 (v1.0.0)

Fecha: Julio 2026

Manual de Usuario: Implementación de PyC45 con Nuevos Datos

Este manual práctico proporciona una guía paso a paso para aplicar el framework PyC45 a cualquier conjunto de datos nuevo de clasificación binaria (por ejemplo, predicción de abandono de clientes, aprobación de créditos, detección de fraudes o diagnósticos médicos).

1. Requisitos de los Datasets

Para que tu dataset sea compatible con PyC45, debe cumplir con las siguientes características de estructura:

1. Clasificación Binaria: La variable objetivo (target) debe estar codificada numéricamente como 0 (clase negativa / no ocurrencia) y 1 (clase positiva / ocurrencia).
 2. Formato Tabular: Los datos deben poder cargarse en un DataFrame de Pandas (formatos .csv, .xlsx, .xls, .tsv, etc.).
 3. Variables Predictoras: Pueden ser numéricas (continuas o discretas) o categóricas codificadas numéricamente (por ejemplo: género como 1 y 2, o nivel de educación como 1, 2, 3).
 4. Sin Valores Nulos (NaN): El algoritmo requiere que limpies o imputes previamente los registros con datos faltantes.
-

2. Guía de Implementación Paso a Paso

A continuación se detalla el flujo de código estructurado para implementar un nuevo caso de uso.

Paso 1: Importar los Módulos de PyC45

Crea un archivo script de Python (ej. mi_analisis.py) e importa los componentes necesarios:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Componentes de PyC45
from core.classifier import C45Classifier
from core.pruning import DecisionTreePruner
from metrics.confusion_matrix import ConfusionMatrix
from metrics.classification_metrics import ClassificationMetrics
from metrics.roc_auc import ROCAUCCalculator
from visualization.feature_importance import FeatureImportance
from visualization.tree_plot import TreePlotter
```

Paso 2: Cargar y Preprocesar el Dataset

Carga tus datos usando Pandas y limpia las columnas irrelevantes (como identificadores correlativos, nombres o fechas tipo texto).

```
# Carga de datos (ejemplo con un archivo CSV)
df = pd.read_csv("ruta/a/tu/nuevo_dataset.csv")

# 1. Eliminar variables que no aportan valor predictivo (IDs, nombres, etc.)
if "ID" in df.columns:
    df = df.drop("ID", axis=1)

# 2. Definir variables predictoras (X) y variable objetivo (y)
target_col = "mi_variable_objetivo" # Reemplazar con el nombre real de tu columna target
X = df.drop(target_col, axis=1)
y = df[target_col]
```

Paso 3: Dividir en Entrenamiento y Validación

Divide el dataset para poder evaluar la capacidad de generalización del árbol en datos no vistos y realizar la poda de forma correcta.

```
# Definir semilla para reproducibilidad
np.random.seed(42)
indices = np.random.permutation(len(df))
train_size = int(len(df) * 0.70) # 70% Entrenamiento, 30% Validación

X_train = X.iloc[indices[:train_size]]
y_train = y.iloc[indices[:train_size]]
X_val = X.iloc[indices[train_size:]]
y_val = y.iloc[indices[train_size:]]

print(f"Registros de Entrenamiento: {X_train.shape[0]}")
print(f"Registros de Validación: {X_val.shape[0]}")
```

> ****Nota sobre el Volumen de Datos:**** Si tu dataset tiene más de 5,000 registros con variables continuas (decimales), se recomienda tomar una muestra aleatoria (ej. `df_sample = df.sample(n=1500, random_state=42)`) antes de la división, ya que la evaluación de umbrales continuos en Python puro es demandante.

Paso 4: Entrenar el Árbol de Decisión C4.5

Configura el clasificador ajustando los límites de crecimiento:

```
# Inicializar el clasificador
clf = C45Classifier(
    max_depth=5,          # Profundidad máxima del árbol
    min_samples_split=10, # Mínimo de muestras en un nodo para permitir una división
    min_gain_ratio=0.01   # Ganancia mínima requerida para justificar un corte
)

# Entrenar
clf.fit(X_train, y_train)
```

```
# Imprimir resumen del árbol en consola
clf.print_tree()
clf.summary()
```

Paso 5: Simplificar el Árbol (Poda / Pruning)

La poda es fundamental para eliminar las ramas débiles que causan sobreajuste (overfitting):

```
# Aplicar Poda por Error Reducido (REP) usando el conjunto de validación
DecisionTreePruner.prune(clf, X_val, y_val)

# Mostrar el árbol final optimizado
print("\n--- Estructura del Árbol Podado ---")
clf.print_tree()
clf.summary()
```

Paso 6: Generar Métricas y Matriz de Confusión

Evalúa el desempeño del árbol podado en el conjunto de validación:

```
# Predecir etiquetas
predicciones = clf.predict(X_val)

# Crear matriz de confusión
cm = ConfusionMatrix.from_predictions(y_val, predicciones)
cm.summary()

# Calcular métricas extendidas
metricas = ClassificationMetrics(cm)
metricas.summary()
```

Paso 7: Analizar la Curva ROC y calcular el AUC

Calcula las probabilidades de predicción y evalúa el área bajo la curva ROC:

```
# Obtener probabilidades de la clase positiva (1)
probabilidades = clf.predict_proba(X_val)[:, 1]

# Calcular ROC-AUC
datos_roc = ROCAUCCalculator.calculate(y_val, probabilidades)
print(f"Área Bajo la Curva ROC (AUC): {datos_roc['auc']:.4f}")

# Graficar y guardar la curva ROC
plt.figure(figsize=(6, 5))
plt.plot(datos_roc["fpr"], datos_roc["tpr"], color="#E67E22", lw=2, label=f"ROC (AUC = {datos_roc['auc']:.4f})")
plt.plot([0, 1], [0, 1], color="#2C3E50", lw=1.5, linestyle="--")
plt.xlabel("Tasa de Falsos Positivos (FPR)")
plt.ylabel("Tasa de Verdaderos Positivos (TPR)")
plt.title("Curva ROC del Modelo")
plt.legend(loc="lower right")
plt.grid(True, linestyle=":", alpha=0.6)
```

```
plt.savefig("curva_roc_modelo.png", dpi=150)
plt.close()
```

Paso 8: Obtener la Importancia de las Variables y Graficar el Árbol

Determina qué variables influyen más en la toma de decisiones y genera la imagen gráfica del árbol:

```
# 1. Extraer importancia de atributos
fi = FeatureImportance().fit(clf.root)
print("\nRanking de importancia de variables:")
print(fi.dataframe())

# 2. Generar y guardar la imagen del árbol estructurado
plotter = TreePlotter()

# Evitar bloqueo de ventana emergente y guardar en disco
original_show = plt.show
plt.show = lambda: None

plotter.plot(clf.root, figsize=(14, 8))
plt.savefig("estructura_del_arbol.png", dpi=150)
plt.close('all')
plt.show = original_show

print("Visualización gráfica del árbol guardada como 'estructura_del_arbol.png'")
```

3. Plantilla de Código Lista para Copiar

Puedes copiar este bloque de código y guardarlo como un script (ej. prediccion_generica.py). Solo necesitas modificar los parámetros de la sección "CONFIGURA AQUÍ TU DATASET".

```
# =====
# SCRIPT DE IMPLEMENTACIÓN PyC45 PARA NUEVOS DATASETS
# =====

import sys
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Asegurar importación de PyC45 (si el script se ejecuta en la raíz)
sys.path.append(os.path.abspath('.'))

from core.classifier import C45Classifier
from core.pruning import DecisionTreePruner
from metrics.confusion_matrix import ConfusionMatrix
from metrics.classification_metrics import ClassificationMetrics
from metrics.roc_auc import ROCAUCCalculator
from visualization.feature_importance import FeatureImportance
from visualization.tree_plot import TreePlotter
```

```

# -----
# CONFIGURA AQUÍ TU DATASET
# -----
RUTA_DATASET = "datasets/customer_churn.csv" # Ruta del archivo (.csv o .xls)
COLUMNA_TARGET = "churn" # Nombre de la columna objetivo (0 o 1)
COLUMNAS_A_ELIMINAR = [] # Lista de columnas irrelevantes a remover (ej. ["ID", "Name"])
MAX_SAMPLES = 1500 # Tamaño máximo de muestra (para datasets muy grandes)
# -----

def main():
    print("=" * 70)
    print("                EJECUCIÓN DE PIPELINE PyC45")
    print("=" * 70)

    # 1. Cargar archivo según su extensión
    print(f"[1] Cargando datos desde: {RUTA_DATASET}")
    ext = os.path.splitext(RUTA_DATASET)[1].lower()
    if ext in ['.xls', '.xlsx']:
        df = pd.read_excel(RUTA_DATASET)
    else:
        df = pd.read_csv(RUTA_DATASET)

    # 2. Limpieza de columnas
    if COLUMNAS_A_ELIMINAR:
        df = df.drop(columns=[col for col in COLUMNAS_A_ELIMINAR if col in df.columns], errors='ignore')

    # 3. Muestreo de control si el archivo es demasiado grande
    if len(df) > MAX_SAMPLES:
        print(f"Dataset grande ({len(df)} filas). Submuestreando a {MAX_SAMPLES} registros...")
        df = df.sample(n=MAX_SAMPLES, random_state=42).reset_index(drop=True)

    # 4. Separar variables y Target
    X = df.drop(columns=[COLUMNA_TARGET])
    y = df[COLUMNA_TARGET]

    # 5. División entrenamiento y validación (70/30)
    np.random.seed(42)
    shuffled = np.random.permutation(len(df))
    split = int(len(df) * 0.70)

    X_train, y_train = X.iloc[shuffled[:split]], y.iloc[shuffled[:split]]
    X_val, y_val = X.iloc[shuffled[split:]], y.iloc[shuffled[split:]]

    print(f"Train size: {X_train.shape[0]} | Validation size: {X_val.shape[0]}")

    # 6. Entrenamiento del Clasificador
    print("\n[2] Entrenando Clasificador C4.5...")
    clf = C45Classifier(max_depth=5, min_samples_split=10, min_gain_ratio=0.01)
    clf.fit(X_train, y_train)

    print("\n--- Árbol Inicial (Sin Podar) ---")
    clf.print_tree()
    print(f"Exactitud en Entrenamiento: {clf.score(X_train, y_train):.4f}")
    print(f"Exactitud en Validación : {clf.score(X_val, y_val):.4f}")

    # 7. Poda del Árbol
    print("\n[3] Ejecutando Poda por Error Reducido (REP)...")
    DecisionTreePruner.prune(clf, X_val, y_val)

    print("\n--- Árbol Final (Podado) ---")
    clf.print_tree()
    clf.summary()

```

```

# 8. Evaluación de rendimiento
print("\n[4] Evaluando Métricas...")
preds = clf.predict(X_val)
cm = ConfusionMatrix.from_predictions(y_val, preds)
cm.summary()

metrics = ClassificationMetrics(cm)
metrics.summary()

# 9. ROC-AUC
probs = clf.predict_proba(X_val)[: , 1]
roc_data = ROCAUCCalculator.calculate(y_val, probs)
print(f"Área Bajo la Curva ROC (AUC): {roc_data['auc']:.4f}")

# 10. Importancia y Exportación de gráficos
print("\n[5] Generando Gráficos de Salida...")
fi = FeatureImportance().fit(clf.root)
print("\nRanking de Importancia:")
print(fi.dataframe())

# Curva ROC PNG
plt.figure(figsize=(6, 5))
plt.plot(roc_data["fpr"], roc_data["tpr"], color="#E67E22", lw=2, label=f"ROC (AUC = {roc_data['auc']:.4f})")
plt.plot([0, 1], [0, 1], color="#2C3E50", lw=1.5, linestyle="--")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.title("Curva ROC de Validación")
plt.legend(loc="lower right")
plt.grid(True, linestyle=":", alpha=0.6)
plt.savefig("salida_curva_roc.png", dpi=150)
plt.close()

# Árbol PNG
plotter = TreePlotter()
original_show = plt.show
plt.show = lambda: None
plotter.plot(clf.root, figsize=(14, 8))
plt.savefig("salida_estructura_arbol.png", dpi=150)
plt.close('all')
plt.show = original_show

print("\n[V] Proceso terminado con éxito.")
print("    - Curva ROC guardada como 'salida_curva_roc.png'")
print("    - Estructura del árbol guardada como 'salida_estructura_arbol.png'")
print("=" * 70)

if __name__ == "__main__":
    main()

```